Problema da moeda

Problema: dado um valor alvo V e um cojunto de valores de moedas $C = [c_1, c_2, ..., c_n]$, determine o menor número de moedas necessárias para formar o valor V.

Já vimos como resolver esse problema antes usando algoritmo guloso com moedas de valores 1, 5, 10, 25, 50 e 100, mas agora veremos como calcular para qualquer conjunto de moedas.

Por exemplo, para $V = 6$ e $C = [1, 3, 4]$, a solução ótima é usar duas moedas de valor 3, ao invés de usar a abordagem gulosa e escolher uma moeda de valor 4 e duas moedas de valor 1.

Solução recursiva

A ideia é, para um dado V, utilizar uma moeda c_i (somando +1 para o número de moedas utilizadas) e resolver o problema recursivamente para o valor $V - c_i$.

No exemplo de $V = 6$ e $C = [1, 3, 4]$, temos:

$$\text{solve}(6) = 1 + \min \begin{cases} \text{solve}(5), \\ \text{solve}(3), \\ \text{solve}(2). \end{cases}$$

Ou, para qualquer valor de V:

$$\text{solve}(V) = 1 + \min \begin{cases} \text{solve}(V-1), \\ \text{solve}(V-3), \\ \text{solve}(V-4). \end{cases}$$

Ou, para qualquer valor de V e conjunto de moedas C:

$$\text{solve}(V) = 1 + \min \begin{cases} \text{solve}(V-c_1), \\ \text{solve}(V-c_2), \\ ... \\ \text{solve}(V-c_n). \end{cases}$$

Além disso, temos dois casos base:

Se $V = 0$, então não precisamos de nenhuma moeda, logo $\text{solve}(0) = 0$. Se V é negativo, então chegamos em um valor impossível de ser formado, logo $\text{solve}(V) = \infty$.

A ideia é usar um valor grande para representar o infinito, como 10^9 ou $V + 1$ (precisa ser maior que qualquer valor alcançado como resposta). Dessa forma, o infinito não afeta o resultado do mínimo, pois qualquer outro valor finito será menor que o infinito ($min(x, \infty) = x$).

Implementação com DP recursiva

Temos a seguinte implementação recursiva com memoização:

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MAXV = 100010;
const int MAXN = 101;
const int INF = 1e9; // 10 ^ 9

int V;
int n, C[MAXN];

int dp[MAXV];

int solve(int v){
    // caso base
    if(v == 0) return 0;
    if(v < 0) return INF;

    // verifica se já foi calculado
    if(dp[v] != -1) return dp[v];

    // armazena o valor calculado
    int ans = INF;
    for(int i = 0; i < n; i++){
        ans = min(ans, 1 + solve(v - C[i]));
    }
    dp[v] = ans;
    return dp[v];
}

int main(){
    for(int i = 0; i < MAXV; i++) dp[i] = -1;

    cin >> n >> V;
    for(int i = 0; i < n; i++){
        cin >> C[i];
    }
    int ans = solve(V);
    if(ans >= INF){
        cout << -1 << endl;
    }
    else{
        cout << ans << endl;
    }
}
```

Complexidade: $O(n \cdot V)$.

Implementação com DP iterativa

Utilizando tabulação, podemos implementar a solução de forma iterativa:

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MAXV = 100010;
const int MAXN = 101;
const int INF = 1e9; // 10 ^ 9

int n, V, C[MAXN];
int dp[MAXV];

int main(){
    cin >> n >> V;
    for(int i = 0; i < n; i++){
        cin >> C[i];
    }

    dp[0] = 0;
    for(int v = 1; v <= V; v++){
        dp[v] = INF;
        for(int i = 0; i < n; i++){
            if(v - C[i] >= 0){
                dp[v] = min(dp[v], 1 + dp[v - C[i]]);
            }
        }
    }

    if(dp[V] >= INF){
        cout << -1 << endl;
    }
    else{
        cout << dp[V] << endl;
    }
}
```

Complexidade: $O(n \cdot V)$.

O problema Minimizing Coins do CSES possui seu limite de tempo bem apertado, fazendo que somente a solução iterativa funcione.